

CMPE 372.02 SAMPLE FULL SUBMISSION REPORT

Group Name

Students' Name
Date

python-docx
[E-posta adresi]

İçindekiler Tablosu

Project Overview.....	3
Personas	4
Persona 1 – Student User.....	4
Persona 2 – Staff User	4
Persona 3 – Campus Security User	5
User Stories.....	6
Student User Stories (Elif).....	6
Staff User Stories (Mehmet)	6
Campus Security User Stories (Ahmet)	6
User Journey	7
Student User Journey Example	7
User Journey Map	8
Information Architecture (IA).....	8
IA Goals:	8
Global Navigation Model:	9
Sitemap (Page Hierarchy)	9
Core Navigation Paths (Task Flows).....	10
Labeling System.....	11
Page Content Inventory.....	11
Staff Panel	12
Security Dashboard.....	12
Site Map	13
Wireframes.....	13
Low Fidelity Screens	16
High Fidelity Screens.....	17
Design System	17
Color System	17
Core UI Components.....	18
States & Microinteractions.....	19
ER Data Model.....	19
Relational Schema	21
Business Rules (ER Constraints)	21

Relationships & Cardinality.....	22
CRUD Matrix	23
SQL Tables	24
Relational Schema Design: Tables + Primary Key /Freign Key + Normalization.....	24
Relationships in Relational Form (PK/FK Summary)	27
Integrity Constraints (Must-have).....	27
API Endpoints.....	28
Async Fetch Flow	29
Accessibility Review	31
Final Reflection.....	33

Project Overview

This project designs and implements a data-driven Lost & Found web application intended for use within a university campus environment. The system provides a centralized digital platform that enables students, university staff, and campus security personnel to efficiently report, search, track, and manage lost and found items.

Traditional lost-and-found processes on campus are often fragmented, manual, and inefficient, relying on physical notice boards, verbal communication, or isolated security offices. These approaches lead to delayed item recovery, poor record-keeping, lack of transparency, and user frustration. The proposed system addresses these issues by offering a structured, searchable, and accessible web-based solution that supports real-time updates and role-based interactions.

The application allows:

- **Students** to report lost items, browse found items, and track the status of their reports.
- **Staff members** (e.g., administrative or facilities staff) to log items found in offices or shared spaces.
- **Campus security** to verify items, manage custody, approve claims, and maintain system integrity.

From a UI/UX perspective, the project emphasizes:

- User-centered design principles
- Clear information architecture
- Minimal cognitive load
- Accessibility and usability across devices

From a technical perspective, the system is designed as a data-driven web application, where:

- User interfaces are mapped directly to underlying data models
- Forms correspond to database entities
- CRUD (Create, Read, Update, Delete) operations support core workflows
- Asynchronous data interactions enable responsive user experiences

The project follows an iterative design and validation process, beginning with user analysis (personas, user stories, journey maps), progressing through low- and high-fidelity prototyping, and culminating in implementation-ready artifacts such as database schemas, CRUD matrices, and API interaction models.

The final outcome is a scalable, maintainable Lost & Found system that demonstrates how thoughtful UI/UX design combined with sound system architecture can deliver meaningful

organizational value, improve operational efficiency, and enhance the overall campus experience.

Personas

Personas were developed to represent the primary user groups of the Lost & Found system and to guide interface, functionality, and data modeling decisions. Three main personas were identified based on typical campus stakeholders: students, staff, and campus security personnel.

Persona 1 – Student User

Name: Elif Kaya
Age: 21
Program: Computer Engineering (Undergraduate)
Technology Comfort Level: High
Goals:

- Quickly report lost personal items
- Browse found items efficiently
- Receive timely updates on claim status
- Recover belongings without visiting multiple offices

Emotion(s):

- Frustration
- Physical lost & found boards are outdated
- No centralized digital tracking
- Long waiting times and lack of communication

Typical Tasks:

- Submit a lost item report
- Search for matching found items
- Upload proof of ownership
- Track claim approval status

System Needs:

- Simple and fast reporting forms
- Keyword-based item search
- Mobile-friendly UI
- Status notifications

Persona 2 – Staff User

Name; Mehmet Arslan
Age: 38
Role: Administrative Office Staff
Technology Comfort Level: Medium
Goals:

- Register found items in his department
- Ensure accurate item descriptions
- Transfer items to campus security efficiently

Emotion(s):

- Frustration
- Manual paper logs
- Risk of item misplacement
- No formal digital record of handovers

Typical Tasks:

- Create found-item entries
- Add location and time metadata
- Mark items as transferred to security

System Needs:

- Simple admin panel
- Clear item logging structure
- Timestamped records
- Ability to update item status

Persona 3 – Campus Security User

Name: Ahmet Demir

Age:45

Role: Campus Security Supervisor

Technology Comfort Level: Medium

Goals:

- Maintain custody of found items
- Verify ownership claims
- Approve item retrievals
- Generate reports

Emotion(s):

- Frustrations
- Lack of centralized tracking
- Time-consuming manual approvals
- Missing item documentation

Typical Tasks:

- Review claim requests
- Approve or reject claims
- Update item status
- Audit historical records

System Needs:

- Dashboard for pending claims
- Verification workflow
- Audit trails
- Role-based permissions

These personas guided:

- Navigation structure

- Form design
- Database entity design
- CRUD workflow definitions
- Validation and access control policies

User Stories

Student User Stories (Elif)

User Story (US) ID	User Story
US1	As a student, I want to submit a lost item report so that my item can be recorded in the system.
US2	As a student, I want to search found items using keywords so that I can locate my lost belongings.
US3	As a student, I want to upload proof of ownership so that I can verify my claim.
US4	As a student, I want to track claim status so that I know whether my item is approved.
US5	As a student, I want to receive notifications so that I am informed about updates.

Staff User Stories (Mehmet)

User Story (US) ID	User Story
US6	As staff, I want to log found items so that they are registered digitally.
US7	As staff, I want to assign found items to campus security so that items are centrally managed.
US8	As staff, I want to update item status so that the system remains accurate.

Campus Security User Stories (Ahmet)

User Story (US) ID	User Story
--------------------	------------

US9	As campus security, I want to review ownership claims so that items are returned securely.
US10	As campus security, I want to approve or reject claims so that unauthorized retrieval is prevented.
US11	As campus security, I want to generate reports so that I can audit system usage.

User Journey

Student User Journey Example

Step	Action	System Response
1	User submits lost item	Lost item saved in database
2	User searches found items	Matching items displayed
3	User submits claim	Claim request created
4	Security reviews claim	Claim approved/rejected
5	User retrieves item	Status updated to "Returned"

Staff User Journey Example

...

Campus Security User Journey Example

...

User Journey Map

Stages	1. Awareness	2. Search	3. Filter	4. View Item
User Tasks	Realizes the item is lost and searches for the official Lost & Found portal online.	Enters relevant keywords to search the portal for their lost item.	Uses filters (Category, Date, Location) to narrow down the search results.	Checks the item details, description, and especially the photo of a potential match.
Emotions	 Frustration/Anxiety: Stress of losing an important item.	 Hopeful Start: Expectation of finding some results.	 Tension/Fatigue: Feeling lost among too many irrelevant results.	 Relief/Excitement: A strong feeling that the item might be theirs.
Pain Points	Unaware of the portal's existence or struggling to find the official link quickly.	The search returns too many irrelevant results due to general keywords.	Filtering options are confusing, too complex, or insufficient (e.g., only precise dates instead of "Yesterday").	The item photo is low quality, blurry, or insufficient details (e.g., model, color) are provided.
Opportunities	Increase Visibility: Use a large, prominent banner/link on the university's homepage or student portal.	Enlarge Main Search: Make the search bar the most prominent element on the landing page. Smart Search: Offer popular category/location suggestions as the user types.	Quick Filters: Introduce pre-set quick filter buttons like "Category," "Last 24 hours," "Last 7 days." Visual Filtering: Offer visual tags (e.g., "Red," "Backpack").	Enforce Clear Photos: Require upload of high-resolution, multiple-angle photos when an item is registered. Large 'Claim Item' CTA: Use a clear Call-to-Action button.

Information Architecture (IA)

IA Goals:

In this project, the information architecture is designed to:

- Reduce cognitive load by grouping tasks into clear modules (Report, Search, Claims, Admin).
- Support role-based navigation (Student/Staff/Security) without hiding core discovery (Found Items).
- Enable fast recovery workflows: Search → Claim → Approval → Pickup.

- Ensure consistency across pages through predictable placement of navigation, filters, and actions.

Global Navigation Model:

Primary Navigation (top nav or side nav)

Common to all roles:

- Home
- Browse Found Items
- Report Lost Item
- My Reports / My Claims
- Help / FAQ
- Login / Profile

Role-specific entry (visible only after login):

Staff Panel (for staff):

- Security Dashboard (for campus security)

Secondary Navigation (within modules)

- Filters, sorting, categories, date range, campus locations
- Tabs for status views (e.g., Pending / Approved / Rejected / Returned)

Sitemap (Page Hierarchy)

Public Area (No login required)

1. Home
 - Quick search bar
 - “Report Lost Item” Click to Action (CTA)
 - “Browse Found Items” link
2. Browse Found Items
 - Item listing (cards/table)
 - Filter & sort
 - Item detail page
3. Found Item Detail
 - Description, location, date/time found
 - Status (e.g., available / claimed / returned)
 - CTA: “Claim this item” → requires login
4. Help / FAQ
 - How the process works
 - What counts as proof
 - Pickup instructions
5. Login / Register

Authenticated Area (Student + Staff)

- 6. Report Lost Item (Form)
 - Required fields + attachments
 - Confirmation page
- 7. My Reports
 - List of submitted lost reports
 - Status tracking
 - Edit / close report
- 8. My Claims
 - Submitted claims
 - Claim status + messages
- 9. Profile
 - Contact info, notification preferences

Staff Panel (Staff Only)

- 10. Log Found Item (Form)
 - “Found in office” workflow
- 11. Department Found Items
 - Items logged by that staff member/department
 - “Transfer to security” action
- 12. Transfer Log
 - Audit list of transferred items

Security Dashboard (Security Only)

- 13. Security Dashboard (Overview)
 - KPIs: pending claims, unverified items, items ready for pickup
- 14. Manage Found Items
 - Verify item entry
 - Update custody status
- 15. Claim Review
 - List of pending claims
 - Claim detail + proof review
 - Approve / Reject / Request more info
- 16. Reports & Audit
 - Export list (CSV/PDF optional)
 - Claim history, item lifecycle history

Core Navigation Paths (Task Flows)

A. Student Recovery Flow:

Home → Browse Found Items → Item Detail → Submit Claim → My Claims → Pickup Confirmation

B. Reporting Flow:

Home → Report Lost Item → Confirmation → My Reports (status updates)

C. Staff Found Item Flow:

Staff Panel → Log Found Item → Department Found Items → Transfer to Security

D. Security Verification & Claim Flow:

Security Dashboard → Claim Review → Approve/Reject → Manage Found Item Status
→ Audit Record

Labeling System

To prevent confusion, labels are consistent and user-friendly:

- Report Lost Item (not “Create record” (no tech lang))
- Browse Found Items (not “Search database”)
- Claim Item (not “Initiate request”)
- My Reports / My Claims
- Status labels:
 - Available
 - Claim Pending
 - Approved
 - Rejected
 - Returned
 - Transferred to Security
- Categories:
 - i.e. Electronics, ID/Cards, Bags, Clothing, Keys, Books, Accessories, Other
- Locations:
 - Building → Floor → Room (or campus zones)

Page Content Inventory

Page	Purpose	Main Content	User Actions
Home	Entry point & guidance	Search bar, CTA buttons, statistics, instructions	Search, Go to Browse, Report Lost
Browse Found Items	Discover found items	Item cards, filters (category, date, location), sort	View details
Found Item Detail	Item inspection	Photos, description, found location/date, status	Claim Item
Report Lost Item	Submit lost reports	Form (category, description, location, image upload)	Submit report

Confirmation Page	Submission feedback	Report ID, next steps	Go to My Reports
My Reports	Track lost submissions	Table of reports, statuses	Edit/Close report
My Claims	Track claim requests	Claim list, decision status	View messages
Help / FAQ	Reduce support need	Process explanations, proof rules	—
Profile	User info	Contact, notification settings	Update info

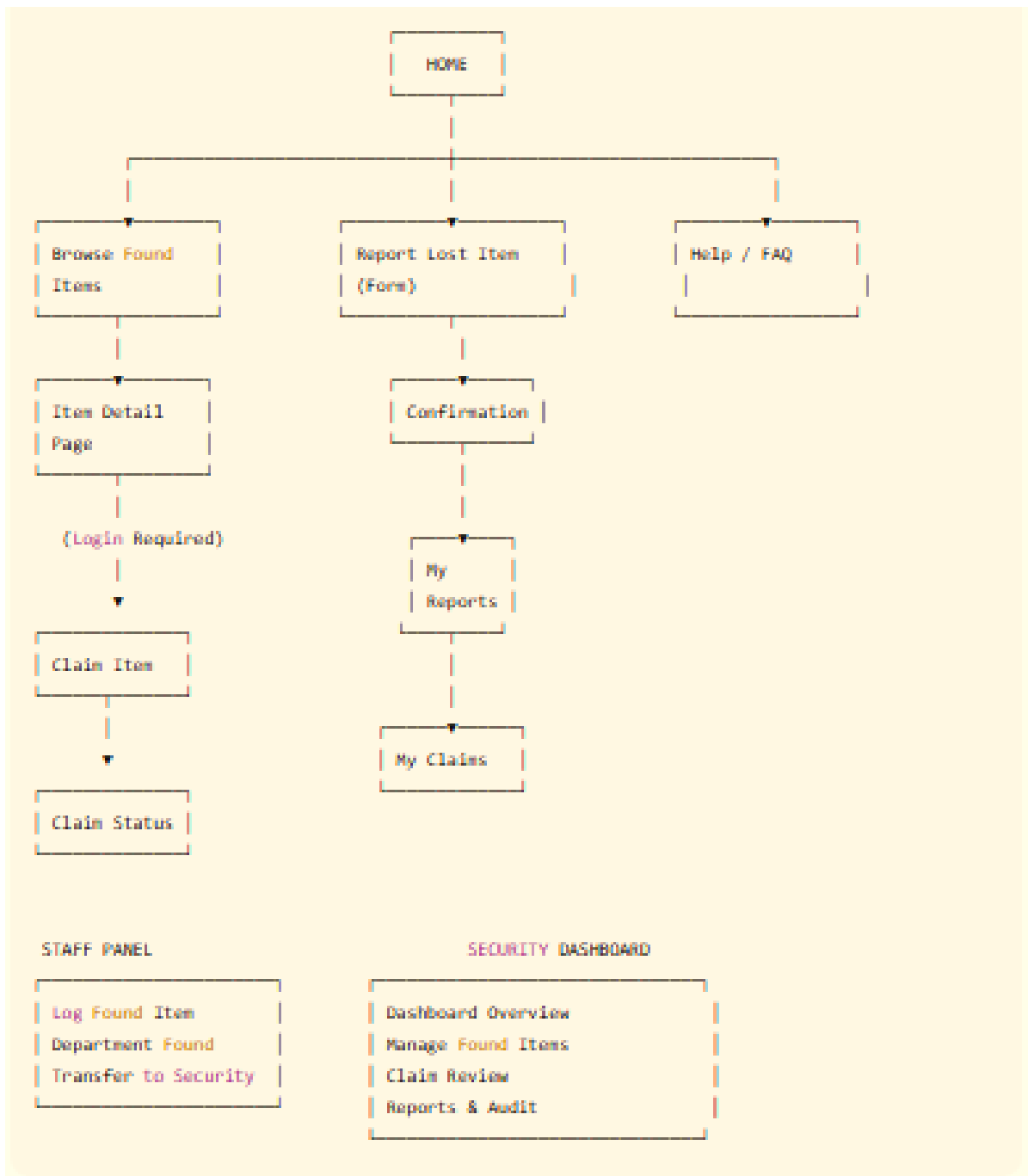
Staff Panel

Page	Purpose	Content	Actions
Log Found Item	Enter found objects	Found item form	Submit found item
Department Found Items	Manage department items	List of found items	Transfer to security
Transfer Log	Traceability	Transfer history	View

Security Dashboard

Page	Purpose	Content	Actions
Dashboard Overview	Monitor system	KPIs, alerts	Navigate
Manage Found Items	Verify custody	Item list, status	Update
Claim Review	Decide claims	Proof review panel	Approve / Reject
Reports & Audit	Compliance	Logs, export	Export CSV/PDF

Site Map



Wireframes

Page Layout Specifications

Global Layout (All Pages)

- Header (Top)

- Logo / App name (click → Home)
 - Primary nav: Home | Browse | Report Lost | My Claims/Reports | Help
 - Right: Login/Profile dropdown
- Main Content
 - Page title + short helper text
 - Breadcrumbs (optional): Home > Browse > Item Detail
 - Content area (responsive grid)
- Footer
 - Contact, campus office hours, privacy policy

Responsive Grid Rules

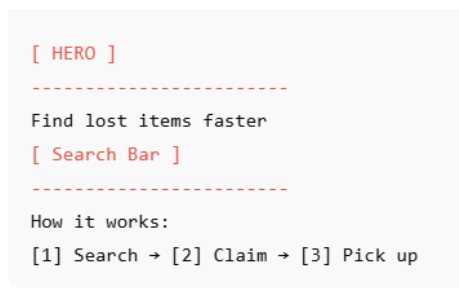
- Desktop: **12-column grid**, max width ~1200px
- Tablet: **8-column**
- Mobile: **4-column / single column stacking**
- Consistent spacing: 8px base (8/16/24/32)

Page Wireframes (Key Screens)

A) Home

Sections

1. Hero: “Find lost items faster” + Search bar
2. Two CTAs (cards): Report Lost / Browse Found
3. “How it works” (3 steps)
4. Latest found items preview (optional)



Primary actions: Search, Browse, Report

B) Browse Found Items

Left panel (Desktop) / Drawer (Mobile)

- Filters: Category, Location, Date range, Status
- Search input

Main panel

- Results count + Sort dropdown (Newest, Oldest, Relevance)
- Item cards grid (image, title, location, date, status)

Primary actions: Open Item Detail

C) Found Item Detail

Top

- Image gallery + item title + status badge

Details

- Found location/time

Description, category, unique identifiers

CTA

- “Claim this item” (requires login)
- Secondary: “Report similar item” (optional)

D) Report Lost Item (Form)

Form blocks

- Item info: category, title, description
- Last seen: location, date/time
- Contact: preferred email/phone
- Upload: image/proof (optional)
- Consent checkbox + Submit

Validation UX

- Inline errors + clear guidance

E) Claim Item (Form)

- Pre-filled item details
- Proof of ownership upload
- Description of proof (text)
- Submit claim

F) My Reports / My Claims (Dashboard)

Tabs

All | Pending | Approved | Rejected | Returned

Table columns

ID, item, date, status, action

Actions

View detail, edit (if pending), close report

G) Security Dashboard

Cards

- Pending claims (#)
- Items awaiting verification (#)
- Ready for pickup (#)

Tables

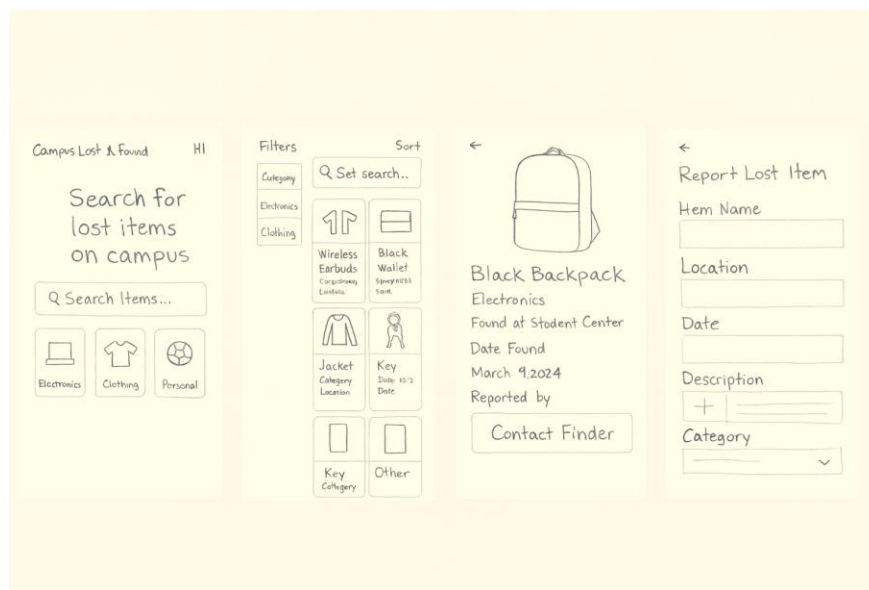
- Pending claims queue (priority by time)

Actions

- Review claim → Approve/Reject/Request more info
- Update item status (Available → Claimed → Returned)

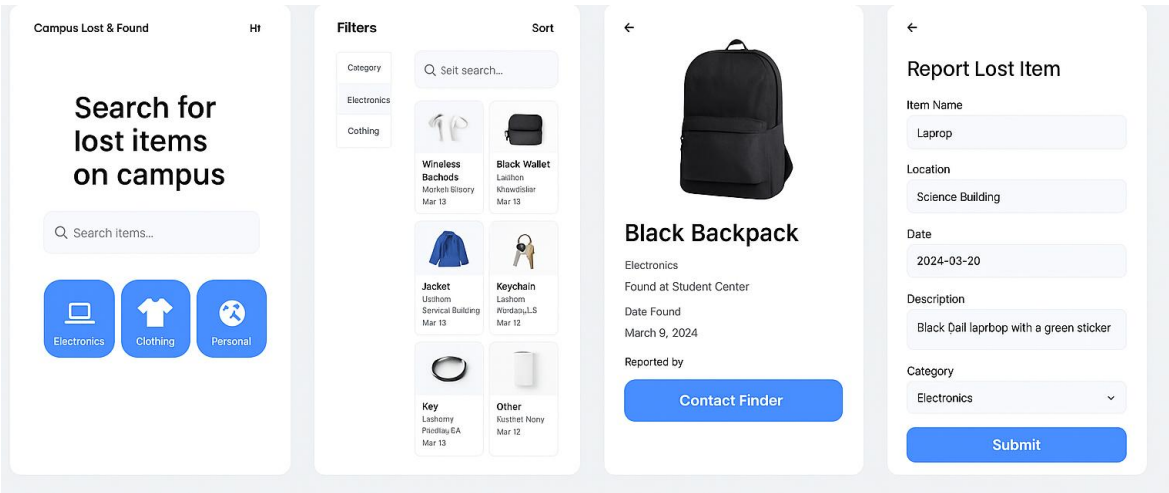
Low Fidelity Screens

- **Homepage:** clear search + categories.
- **Item List:** modern card design + filters.
- **Detail Page:** hierarchy + CTA button.



High Fidelity Screens

- **Homepage:** clear search + categories.
- **Item List:** modern card design + filters.
- **Detail Page:** hierarchy + CTA button.



Design System

This section documents the unified design system created for the Lost & Found Campus Web Application. The design system ensures visual consistency, usability, accessibility, and scalability across all screens and future extensions of the system.

Color System

Purpose	Color Name	HEX
Primary Action	Campus Blue	#1E88E5
Success	Green	#43A047
Warning	Orange	#FB8C00
Error	Red	#E53935
Background	Light Gray	#F5F5F5
Text Primary	Dark Gray	#212121

Colors were selected to provide sufficient contrast ratios to meet WCAG 2.1 AA accessibility requirements.

Typography

Usage	Font	Size
Headings	Inter / Roboto	32-18 px
Body Text	Inter / Roboto	16 px
Labels & Buttons	Inter / Roboto	14 px

Typography supports:

- Clear hierarchy
- Mobile readability
- Accessibility compliance

Spacing & Grid System

- Base spacing unit: 8px
- 12-column responsive grid
- Breakpoints:
 - Mobile: <768px
 - Tablet: 768-1024px
 - Desktop: >1024px

This ensures consistency and responsive behavior across devices.

Core UI Components

Component	Description
Buttons	Primary, Secondary, Destructive
Input Fields	Text, dropdown, file upload
Cards	Item preview and result display
Badges	Status indicators (Lost, Found, Claimed)
Modals	Claim confirmation & alerts
Navigation Bar	Global navigation
Breadcrumbs	Page hierarchy indication

Alerts	Success, error, warning messages
---------------	----------------------------------

States & Microinteractions

State	Behavior
Hover	Highlight border & shadow
Focus	Blue outline (keyboard accessibility)
Loading	Spinner animation
Error	Red border & helper text
Success	Green confirmation message

Microinteractions provide immediate system feedback and improve perceived usability.

The Lost & Found Design System provides a scalable, accessible, and consistent visual foundation for the application. It ensures that future development and new features can be integrated without breaking usability, accessibility, or visual identity.

ER Data Model

Core Entities (High-Level)

Entity 1: User

Represents any person using the system (student, staff, campus security). Role is crucial for access control (e.g., security can approve claims; students cannot).

Key Attributes

- user_id (PK)
- role (Student / Staff / Security / Admin)
- full_name
- email (unique)
- phone (optional)
- created_at

Entity 2: Item

Represents a lost or found object.

Key Attributes

- item_id (PK)
- title (e.g., “Black Wallet”)
- description
- category (ID card, electronics, clothing, keys...)
- item_type (Lost / Found)
- status (Open, Claimed, Verified, Returned, Archived)
- date_reported
- date_lost_or_found (optional)
- photo_url (optional)
- location_id (FK reference conceptually)
- reported_by (FK → User)

reported_by is mandatory: every item is created by a user.

status changes over time; we track changes in AuditLogs.

Entity 3: Location

Standardizes where an item was lost/found.

Key Attributes

- location_id (PK)
- building
- campus_zone
- details (optional: “near cafeteria entrance”)

Entity 4: Claim

Represents a user claiming an item.

Key Attributes

- claim_id (PK)
- item_id (FK → Item)
- claimer_id (FK → User)
- claim_reason / proof_description
- claim_status (Pending, Approved, Rejected, Cancelled)
- submitted_at
- reviewed_by (FK → User, optional)
- reviewed_at (optional)

One item can receive multiple claims, but only one can be approved.

Claim connects “people” and “items” in a controlled workflow.

Entity 5: Notification

Represents system messages (status updates, approvals, reminders).

Key Attributes

- notification_id (PK)
- user_id (FK → User)
- type (StatusChange, ClaimUpdate, Reminder)
- message
- is_read
- created_at

Entity 6: AuditLog

Tracks critical actions for accountability. This entity is important for

- Debugging, compliance, security monitoring
- “Who changed what and when?” is required in real systems

Key Attributes

- log_id (PK)
- actor_user_id (FK → User)
- action_type (CREATE_ITEM, UPDATE_STATUS, APPROVE_CLAIM...)
- target_type (Item / Claim / User)
- target_id
- timestamp
- details (optional)

Relational Schema

Business Rules (ER Constraints)

These are rules you can explicitly write under your ER diagram:

1. An item must have exactly one reporter.
2. A claim must reference exactly one item and one claimer.
3. An item may receive multiple claims, but at most one claim can be approved.
4. Approved claim implies item status moves toward “Claimed/Verified/Returned”.
5. Only users with role Security/Staff can review claims.

6. Audit logs must be created for key actions: item creation, status change, claim approval/rejection.

Relationships & Cardinality

- R1: User — reports — Item
- 1 User can report 0 ... N Items
- Each Item must be reported by exactly 1 User

Cardinality: User (1) → Item (M)

- R2: Location — contains — Item
- 1 Location can contain 0 ... N Items
- Each Item is associated with 0 ... 1 Location
- It is optional, because some users may not know exact location

Cardinality: Location (1) → Item (M), optional on Item side

- R3: User — submits — Claim
 - 1 User can submit 0...N Claims
 - Each Claim belongs to exactly 1 User

Cardinality: User (1) → Claim (M)

- R4: Item — has — Claim
 - 1 Item can have 0...N Claims
 - Each Claim is linked to exactly 1 Item

Cardinality: Item (1) → Claim (M)

- R5: User (Security/Staff) — reviews — Claim
- 1 Claim may be reviewed by 0...1 User (reviewer)
- 1 Reviewer can review 0..N Claims

Cardinality: User (1) → Claim (M), optional on Claim side

- R6: User — receives — Notification
- 1 User receives 0...N Notifications
- Each Notification belongs to exactly 1 User

Cardinality: User (1) → Notification (M)

- R7: User — creates — AuditLog
- 1 User creates 0...N logs
- Each AuditLog belongs to exactly 1 actor

Cardinality: User (1) → AuditLog (M)

CRUD Matrix

UI Screens → Tables → CRUD

Core Tables

- Users
- LostItems
- FoundItems
- Claims
- Transfers
- Notifications (optional)
- AuditLogs (CE recommended)

UI Screen / Feature	Users	LostItems	FoundItems	Claims	Transfers	AuditLogs
Register/Login	C/R/U	-	-	-	-	C
Report Lost Item	R	C/R/U	-	-	-	C
Browse Found Items	-	-	R	-	-	C
Found Item Detail	-	-	R	-	-	C
Claim Item	R	-	R	C/R	-	C
My Reports	-	R/U	-	-	-	C
My Claims	-	-	R	R	-	C

Staff Log Found Item	R	-	C/R/U	-	-	C
Transfer to Security	R	-	U	-	C/R	C
Security Claim Review	R	-	U	R/U	R	C/R
Mark Returned	-	-	U	U	-	C
Reports & Audit	-	R	R	R	R	R

SQL Tables

Relational Schema Design: Tables + Primary Key /Freign Key + Normalization

Tables

Users

Column	Type	Constraints
user_id	INT / UUID	PK
role	VARCHAR	NOT NULL, CHECK(role IN ...)
full_name	VARCHAR	NOT NULL
email	VARCHAR	NOT NULL, UNIQUE
phone	VARCHAR	NULL
created_at	TIMESTAMP	NOT NULL (default now)

Locations

Column	Type	Constraints
location_id	INT / UUID	PK
building	VARCHAR	NOT NULL, UNIQUE

campus_zone	VARCHAR	NOT NULL, UNIQUE
details	VARCHAR	NULL

Items

Column	Type	Constraints
item_id	INT / UUID	PK
title	VARCHAR	NOT NULL
description	TEXT	NOT NULL
category	VARCHAR	NOT NULL
item_type	VARCHAR	NOT NULL, CHECK(item_type IN ('Lost','Found'))
status	VARCHAR	NOT NULL, CHECK(status IN ('Open','Claimed','Verified','Returned','Archived'))
date_reported	DATE	NOT NULL
date_lost_or_found	DATE	NULL
photo_url	VARCHAR	NULL
location_id	INT/UUID	FK → Locations(location_id), NULL allowed
reported_by	INT/UUID	FK → Users(user_id), NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL

Claims

Column	Type	Constraints
claim_id	INT / UUID	PK

item_id	INT/UUID	FK → Items(item_id) NOT NULL
claimer_id	INT/UUID	FK → Users(user_id) NOT NULL
claim_reason	TEXT	NOT NULL
proof_description	TEXT	NULL
claim_status	VARCHAR	NOT NULL, CHECK IN ('Pending','Approved','Rejected','Cancelled')
submitted_at	TIMESTAMP	NOT NULL
reviewed_by	INT/UUID	FK → Users(user_id) NULL
reviewed_at	TIMESTAMP	NULL

E) (Recommended) ItemStatusHistory

Purpose: Item status değişimlerini tarihçelemek (audit'in bir alt türü)

Column	Type	Constraints
history_id	INT/UUID	PK
item_id	INT/UUID	FK → Items(item_id) NOT NULL
old_status	VARCHAR	NOT NULL
new_status	VARCHAR	NOT NULL
changed_by	INT/UUID	FK → Users(user_id) NOT NULL
changed_at	TIMESTAMP	NOT NULL
note	VARCHAR	NULL

Notifications

Column	Type	Constraints
notification_id	INT/UUID	PK

user_id	INT/UUID	FK → Users(user_id) NOT NULL
type	VARCHAR	NOT NULL
message	TEXT	NOT NULL
is_read	BOOLEAN	NOT NULL (default false)
created_at	TIMESTAMP	NOT NULL

AuditLogs

Column	Type	Constraints
log_id	INT/UUID	PK
actor_user_id	INT/UUID	FK → Users(user_id) NOT NULL
action_type	VARCHAR	NOT NULL
target_type	VARCHAR	NOT NULL
target_id	VARCHAR / INT	NOT NULL
timestamp	TIMESTAMP	NOT NULL
details	JSON/TEXT	NULL

Relationships in Relational Form (PK/FK Summary)

- Items.reported_by → Users.user_id (mandatory)
- Items.location_id → Locations.location_id (optional)
- Claims.item_id → Items.item_id (mandatory)
- Claims.claimer_id → Users.user_id (mandatory)
- Claims.reviewed_by → Users.user_id (optional)
- Notifications.user_id → Users.user_id
- AuditLogs.actor_user_id → Users.user_id
- ItemStatusHistory.item_id → Items.item_id

Integrity Constraints (Must-have)

- Users.email UNIQUE

- Items.item_type ve Items.status CHECK constraint (phone, wallet, earbud are fine but pizza, banana, girl friend NOT or for status of an item lost, found, returned OK))
- Claims.claim_status CHECK constraint (pending, approved, rejected)
- Approved claim uniqueness (approved only one claim per item)
- delete/update for FK
 - Items.reported_by → RESTRICT
 - Claims.item_id → RESTRICT + soft delete
 - Notifications.user_id → CASCADE

API Endpoints

Authentication

- POST /api/auth/register → create user
- POST /api/auth/login → session/token
- POST /api/auth/logout

Found Items

- GET /api/found-items (filters: q, category, location, status, dateFrom/dateTo)
- GET /api/found-items/{id}
- POST /api/found-items (*staff/security*)
- PUT /api/found-items/{id} (*update status, details*)

Lost Items

- POST /api/lost-items (*student*)
- GET /api/lost-items/mine (*student's reports*)
- PUT /api/lost-items/{id} (*edit pending*)
- PUT /api/lost-items/{id}/close

Claims

- POST /api/claims (*student claim submission*)
- GET /api/claims/mine (*student*)
- GET /api/claims/pending (*security*)
- GET /api/claims/{id} (*security view proof*)
- PUT /api/claims/{id}/approve

- PUT /api/claims/{id}/reject
- PUT /api/claims/{id}/request-info (*optional*)

Transfers

- POST /api/transfers (*staff* → *security*)
- GET /api/transfers (*security/staff* filterable)

Notifications (optional)

- GET /api/notifications/mine
- POST /api/notifications (*system trigger*)

Audit Logs (CE recommended)

- GET /api/audit (*security/admin*)
- Automatically create on sensitive actions: approve/reject/return/update

Async Fetch Flow

System Architecture Detail

This section describes how the Lost & Found web application retrieves, updates, and synchronizes data between the user interface and the database using asynchronous communication.

Why Asynchronous Fetch?

The system uses asynchronous API communication so that:

- Pages do not reload while data is being loaded
- Users experience fast and smooth interactions
- Multiple users can interact with the system simultaneously
- The UI stays responsive even during network delays

High-Level Flow

User Action → Frontend (JS) → API Request (Fetch) → Backend → Database

↓

JSON Response

↓

UI Updates in Real Time

Core Fetch Architecture

Layer	Technology	Responsibility
Frontend	HTML / CSS / JavaScript	UI rendering & user interactions
Communication	Fetch API (async/await)	Sends & receives JSON
Backend	Flask / Express	Processes logic
Database	PostgreSQL / MySQL	Stores permanent data

Example Async Flow: Reporting a Lost Item

1. User submits the form

```
document.getElementById("reportForm").addEventListener("submit", async e => {  
    e.preventDefault();
```

2. Frontend sends data asynchronously

```
const response = await fetch("/api/items", {  
    method: "POST",  
    headers: {"Content-Type": "application/json"},  
    body: JSON.stringify(formData)  
});
```

3. Backend processes and stores data

```
@app.route("/api/items", methods=["POST"])  
  
def create_item():  
    db.insert(item_data)  
    return jsonify({"status": "success"})
```

4. UI updates without page reload

```
if(response.ok){  
    showToast("Item reported successfully");  
    refreshItemList();
```

```
}
```

Real-Time UI Rendering Example

```
const data = await fetch("/api/items").then(res => res.json());
```

```
renderItems(data);
```

Items appear instantly without reloading the page.

Async Flow for Claim Review

Action	HTTP	Endpoint	Result
Load claims	GET	/api/claims	Loads pending claims
Approve claim	PUT	/api/claims/{id}/approve	Updates DB
UI refresh	GET	/api/claims	Shows updated list

Error Handling (Async Safety)

```
try {  
  const res = await fetch(url);  
  if(!res.ok) throw new Error("Server Error");  
} catch(e){  
  showError("Network issue. Try again.");  
}
```

Accessibility Review

Accessibility was treated as a core design requirement rather than an optional enhancement. The application meets WCAG 2.1 AA guidelines and supports inclusive use across diverse user groups, ensuring equitable digital access within the campus environment. This web portal is designed to:

- Provide equitable access to all users
- Support assistive technologies such as screen readers
- Enable full keyboard navigation
- Reduce cognitive and visual load
- Comply with WCAG 2.1 Level AA standards

WCAG 2.1 Compliance Mapping

WCAG Principle	Implementation
Perceivable	High contrast colors, readable fonts, alt text for images
Operable	Full keyboard navigation, visible focus indicators
Understandable	Clear labels, simple error messages, consistent layout
Robust	Compatible with screen readers and assistive technologies

Visual Accessibility

- Minimum contrast ratio of 4.5:1 maintained
- Text resizing up to 200% supported
- No color-only meaning communication
- Clear hierarchy via typography and spacing

Keyboard Accessibility

- All interactive elements reachable via Tab key
- Logical focus order
- Skip-to-content links
- Focus outlines for all inputs and buttons

Screen Reader Support

- Semantic HTML elements (<nav>, <main>, <section>)
- ARIA labels for form fields
- Alt text for item images
- Form validation feedback announced to assistive technologies

Cognitive Accessibility

- Simple sentence structure
- Clear task flows
- Step-by-step instructions
- Minimal memory load for multi-step tasks

Error Prevention & Recovery

Feature	Purpose
Inline validation	Prevent incorrect data entry
Confirmation dialogs	Avoid accidental submissions

Undo options	Allow recovery from mistakes
--------------	------------------------------

Final Reflection

Working on the Campus Lost & Found Web Application has been one of the most valuable learning experiences I have had in this course. At the beginning of the semester, I thought UI/UX design was mainly about making screens “look nice.” Through this project, I realized that good interface design is actually about understanding people, defining problems clearly, and designing systems that support real user goals in a structured and measurable way.

One of the most important things I learned was how much impact early-stage design decisions have on the final system. Creating personas, user journeys, and task flows helped me understand students, staff, and campus security not just as “users,” but as people with different motivations, constraints, and expectations. These artifacts guided all later design choices, from navigation structure to form layouts and validation rules.

Designing the information architecture taught me how to organize content logically so that users can find what they need without thinking too much. I became more aware of cognitive load, consistency, and recognition-over-recall principles. Small changes, such as button placement, labeling, and grouping of information, made a big difference in how easy the system felt to use.

The usability testing phase was especially eye-opening. Using low-fidelity paper prototypes showed me that we can identify major usability problems long before writing any code. Watching real users hesitate, misunderstand labels, or take unexpected paths helped me understand why testing is not optional but essential in professional design practice.

On the technical side, mapping UI actions to database entities through CRUD matrices and relational schema design helped me see the connection between interface design and system architecture. I now understand how front-end decisions directly affect data integrity, validation logic, and backend implementation.

Overall, this project taught me how to approach interface design as a structured engineering and design process rather than just a visual task. I feel more confident in designing user-centered systems, documenting design decisions, and communicating requirements to developers. Most importantly, I now understand how thoughtful UI/UX design can directly improve system effectiveness and user satisfaction in real organizational contexts.